

- [25] G. Plotkin and M. Pretnar, “Handlers of Algebraic Effects,” in *Programming Languages and Systems (ESOP)*, Berlin, Heidelberg: Springer Berlin Heidelberg, 2009, pp. 80–94. doi: 10.1007/978-3-642-00590-9_7. 1
- [26] M. Pretnar, “An Introduction to Algebraic Effects and Handlers. Invited tutorial paper,” *Electron. Notes Theor. Comput. Sci.*, vol. 319, no. C, pp. 19–35, Dec. 2015, doi: 10.1016/j.entcs.2015.12.003. 2
- [27] D. Leijen, “Koka: Programming with Row Polymorphic Effect Types,” *Electronic Proceedings in Theoretical Computer Science*, vol. 153, pp. 100–126, Jun. 2014, doi: 10.4204/eptcs.153.8. 3
- [28] D. Leijen, “Type directed compilation of row-typed algebraic effects,” in *Proceedings of the 44th ACM SIGPLAN Symposium on Principles of Programming Languages*, in POPL '17. Paris, France: Association for Computing Machinery, 2017, pp. 486–499. doi: 10.1145/3009837.3009872. 4
- [29] A. Bauer and M. Pretnar, “Programming with algebraic effects and handlers,” *Journal of Logical and Algebraic Methods in Programming*, vol. 84, no. 1, pp. 108–123, Jan. 2015, doi: 10.1016/j.jlamp.2014.02.001. 5
- [30] K. Sivaramakrishnan *et al.*, “Retrofitting parallelism onto OCaml,” *Proc. ACM Program. Lang.*, vol. 4, no. ICFP, Aug. 2020, doi: 10.1145/3408995. 6
- [31] T. Petricek, D. Orchard, and A. Mycroft, “Coeffects: a calculus of context-dependent computation,” in *Proceedings of the 19th ACM SIGPLAN International Conference on Functional Programming*, in ICFP '14. Gothenburg, Sweden: Association for Computing Machinery, 2014, pp. 123–135. doi: 10.1145/2628136.2628160. 7
- [32] T. Uustalu and V. Vene, “Comonadic Notions of Computation,” *Electronic Notes in Theoretical Computer Science*, vol. 203, no. 5, pp. 263–284, 2008, doi: 10.1016/j.entcs.2008.05.029. 8
- [33] A. Brunel, M. Gaboardi, D. Mazza, and S. Zdancewic, “A Core Quantitative Coeffect Calculus,” in *Proceedings of the 23rd European Symposium on Programming Languages and Systems - Volume 8410*, Berlin, Heidelberg: Springer-Verlag, 2014, pp. 351–370. doi: 10.1007/978-3-642-54833-8_19. 9
- [34] J. Reed and B. C. Pierce, “Distance makes the types grow stronger: a calculus for differential privacy,” *SIGPLAN Not.*, vol. 45, no. 9, pp. 157–168, Sep. 2010, doi: 10.1145/1932681.1863568. 10
- [35] M. Abadi, A. Banerjee, N. Heintze, and J. G. Riecke, “A core calculus of dependency,” in *Proceedings of the 26th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, in POPL '99. San Antonio, Texas, USA: Association for Computing Machinery, 1999, pp. 147–160. doi: 10.1145/292540.292555. 11
- [36] D. E. Denning, “A lattice model of secure information flow,” *Commun. ACM*, vol. 19, no. 5, pp. 236–243, May 1976, doi: 10.1145/360051.360056. 12
- [37] U. Dal Lago and F. Gavazzo, “A relational theory of effects and coeffects,” *Proc. ACM Program. Lang.*, vol. 6, no. POPL, Jan. 2022, doi: 10.1145/3498692. 13
- [38] R. P. James and A. Sabry, “Yield: Mainstream Delimited Continuations,” in *First International Workshop on the Theory and Practice of Delimited Continuations (TPDC 2011)*, 2011, pp. 20–32. [Online]. Available: <https://homes.luddy.indiana.edu/sabry/files/yield.pdf> 14

- [39] M. Fowler, “Inversion of Control Containers and the Dependency Injection pattern.” ¹
[Online]. Available: <https://martinfowler.com/articles/injection.html>
- [40] A. M. Pitts and I. D. B. Stark, “Observable Properties of Higher Order Functions that Dynamically Create Local Names, or What's New?,” in *Mathematical Foundations of Computer Science 1993 (MFCS 1993)*, in Lecture Notes in Computer Science, vol. 711. Springer, 1993, pp. 122–141. doi: 10.1007/3-540-57182-5_8. ²
- [41] G. D. Plotkin, “LCF Considered as a Programming Language,” *Theoretical Computer Science*, vol. 5, no. 3, pp. 223–255, 1977, doi: 10.1016/0304-3975(77)90044-5. ³
- [42] D. R. Ghica, K. Muroya, and T. Waugh Ambridge, “A Robust Graph-Based Approach to Observational Equivalence,” *Logical Methods in Computer Science*, vol. 21, no. 2, p. 8:1–8:95, 2025, doi: 10.46298/LMCS-21(2:8)2025. ⁴
- [43] M. Shapiro, N. Preguiça, C. Baquero, and M. Zawirski, “A Comprehensive Study of Convergent and Commutative Replicated Data Types,” technical report RR-7506, 2011. ⁵
- [44] X. Leroy and S. Blazy, “Formal Verification of a C-like Memory Model and Its Uses for Verifying Program Transformations,” *Journal of Automated Reasoning*, vol. 41, no. 1, pp. 1–31, 2008, doi: 10.1007/s10817-008-9099-0. ⁶
- [45] A. T. Clements, M. F. Kaashoek, N. Zeldovich, R. T. Morris, and E. Kohler, “The Scalable Commutativity Rule: Designing Scalable Software for Multicore Processors,” in *Proceedings of the 24th ACM Symposium on Operating Systems Principles*, 2013, pp. 1–17. doi: 10.1145/2517349.2522712. ⁷
- [46] A. W. Mazurkiewicz, “Trace Theory,” in *Petri Nets: Applications and Relationships to Other Models of Concurrency, Advances in Petri Nets 1986, Part II*, in Lecture Notes in Computer Science, vol. 255. Springer, 1987, pp. 279–324. doi: 10.1007/3-540-17906-2_30. ⁸
- [47] U. A. Acar, G. E. Blelloch, and R. Harper, “Adaptive functional programming,” *ACM Transactions on Programming Languages and Systems*, vol. 28, no. 6, pp. 990–1034, 2006, doi: 10.1145/1186632.1186634. ⁹
- [48] webpack, “Hot Module Replacement.” [Online]. Available: <https://webpack.js.org/api/hot-module-replacement/> ¹⁰
- [49] Vite, “HMR API.” [Online]. Available: <https://vite.dev/guide/api-hmr> ¹¹
- [50] E. N. (M. Elnozahy, L. Alvisi, Y.-M. Wang, and D. B. Johnson, “A Survey of Rollback-Recovery Protocols in Message-Passing Systems,” *ACM Computing Surveys*, vol. 34, no. 3, pp. 375–408, 2002, doi: 10.1145/568522.568525. ¹²
- [51] H. Garcia-Molina and K. Salem, “Sagas,” in *Proceedings of the 1987 ACM SIGMOD International Conference on Management of Data*, in SIGMOD '87. 1987, pp. 249–259. doi: 10.1145/38713.38742. ¹³
- [52] OSGi Alliance, *OSGi Core Release 8*. OSGi Alliance, 2020. [Online]. Available: <https://docs.osgi.org/specification/osgi.core/8.0.0/> ¹⁴
- [53] J. Kramer and J. Magee, “The Evolving Philosophers Problem: Dynamic Change Management,” *IEEE Transactions on Software Engineering*, vol. 16, no. 11, pp. 1293–1306, 1990, doi: 10.1109/32.60317. ¹⁵

- [54] Y. Vandewoude, P. Ebraert, Y. Berbers, and T. D'Hondt, "Tranquility: A Low Disruptive Alternative to Quiescence for Ensuring Safe Dynamic Updates," *IEEE Transactions on Software Engineering*, vol. 33, no. 12, pp. 856–868, 2007, doi: 10.1109/tse.2007.70733. 1
- [55] J. S. Rellermeier, G. Alonso, and T. Roscoe, "R-OSGi: Distributed Applications Through Software Modularization," in *Proceedings of the ACM/IFIP/USENIX 8th International Middleware Conference*, in *Middleware '07*. 2007, pp. 1–20. doi: 10.1007/978-3-540-76778-7_1. 2
- [56] J. B. Dennis and E. C. Van Horn, "Programming Semantics for Multiprogrammed Computations," *Communications of the ACM*, vol. 9, no. 3, pp. 143–155, 1966, doi: 10.1145/365230.365252. 3
- [57] M. S. Miller, K.-P. Yee, and J. Shapiro, "Capability Myths Demolished," technical report SRL2003–2, 2003. [Online]. Available: <http://zesty.ca/capmyths/usenix.pdf> 4
- [58] R. N. M. Watson, J. Anderson, B. Laurie, and K. Kennaway, "Capsicum: Practical Capabilities for UNIX," in *Proceedings of the 19th USENIX Security Symposium*, 2010, pp. 29–46. [Online]. Available: https://www.usenix.org/legacy/events/sec10/tech/full_papers/Watson.pdf 5
- [59] R. Wahbe, S. Lucco, T. E. Anderson, and S. L. Graham, "Efficient Software-Based Fault Isolation," in *Proceedings of the 14th ACM Symposium on Operating Systems Principles*, in *SOSP '93*. 1993, pp. 203–216. doi: 10.1145/168619.168635. 6
- [60] A. Barth, A. P. Felt, P. Saxena, and A. Boodman, "Protecting Browsers from Extension Vulnerabilities," in *Proceedings of the 17th Annual Network and Distributed System Security Symposium*, in *NDSS '10*. 2010. [Online]. Available: <https://www.ndss-symposium.org/ndss2010/protecting-browsers-extension-vulnerabilities/> 7
- [61] W. W. Ho and R. A. Olsson, "An Approach to Genuine Dynamic Linking," *Software: Practice and Experience*, vol. 21, no. 4, pp. 375–390, 1991, doi: 10.1002/SPE.4380210404. 8
- [62] P. Wadler and S. Blott, "How to Make Ad-hoc Polymorphism Less Ad Hoc," in *Proceedings of the 16th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, in *POPL '89*. 1989, pp. 60–76. doi: 10.1145/75277.75283. 9
- [63] F. Klock and N. Matsakis, "The Rust Language and Type System," in *ACM SIGPLAN ML Family Workshop*, Gothenburg, Sweden, Sep. 2014. 10
- [64] D. Dreyer, R. Harper, M. M. T. Chakravarty, and G. Keller, "Modular Type Classes," in *Proceedings of the 34th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, in *POPL '07*. 2007, pp. 63–70. doi: 10.1145/1190216.1190229. 11
- [65] Microsoft, "Declaration Merging." [Online]. Available: <https://www.typescriptlang.org/docs/handbook/declaration-merging.html> 12
- [66] T. Van Cutsem and M. S. Miller, "Proxies: Design Principles for Robust Object-oriented Intercession APIs," in *Proceedings of the 6th Symposium on Dynamic Languages*, in *DLS '10*. 2010, pp. 59–72. doi: 10.1145/1869631.1869638. 13
- [67] R. Hettinger, "Descriptor HowTo Guide." [Online]. Available: <https://docs.python.org/3/howto/descriptor.html> 14

- [68] P. Maes, "Concepts and Experiments in Computational Reflection," in *Conference on Object-Oriented Programming Systems, Languages, and Applications (OOPSLA)*, 1987, pp. 147–155. doi: 10.1145/38765.38821. 1
- [69] G. Bracha and D. M. Ungar, "Mirrors: design principles for meta-level facilities of object-oriented programming languages," in *Proceedings of the 19th Annual ACM SIGPLAN Conference on Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA)*, 2004, pp. 331–344. doi: 10.1145/1028976.1029004. 2
- [70] R. Rouvoy and P. Merle, "Leveraging component-based software engineering with Fraclet," *Annals of Telecommunications*, vol. 64, no. 1–2, pp. 65–79, 2009, doi: 10.1007/s12243-008-0072-z. 3
- [71] E. Burmako, "Scala Macros: Let Our Powers Combine!," in *Proceedings of the 4th Workshop on Scala*, in SCALA@ECOOP '13. 2013, p. 3:1–3:10. doi: 10.1145/2489837.2489840. 4
- [72] S. Raemaekers, A. van Deursen, and J. Visser, "Semantic Versioning and Impact of Breaking Changes in the Maven Repository," *Journal of Systems and Software*, vol. 129, pp. 140–158, 2017, doi: 10.1016/j.jss.2016.04.008. 5
- [73] P. Lam, J. Dietrich, and D. J. Pearce, "Putting the Semantics into Semantic Versioning," in *Proceedings of the 2020 ACM SIGPLAN International Symposium on New Ideas, New Paradigms, and Reflections on Programming and Software*, in Onward! '20. 2020, pp. 157–179. doi: 10.1145/3426428.3426922. 6
- [74] P. Abate, R. Di Cosmo, R. Treinen, and S. Zacchiroli, "Dependency Solving: A Separate Concern in Component Evolution Management," *Journal of Systems and Software*, vol. 85, no. 10, pp. 2228–2240, 2012, doi: 10.1016/j.jss.2012.02.018. 7
- [75] L. Cardelli, "Structural Subtyping and the Notion of Power Type," in *Proceedings of the 15th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, in POPL '88. 1988, pp. 70–79. doi: 10.1145/73560.73566. 8
- [76] B. Meyer, "Applying "Design by Contract"," *Computer*, vol. 25, no. 10, pp. 40–51, 1992, doi: 10.1109/2.161279. 9
- [77] B. C. Pierce, "Bounded Quantification is Undecidable," *Information and Computation*, vol. 112, no. 1, pp. 131–165, 1994, doi: 10.1006/inco.1994.1055. 10
- [78] A. Haas *et al.*, "Bringing the web up to speed with WebAssembly," in *Proceedings of the 38th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, ACM, 2017, pp. 185–200. doi: 10.1145/3062341.3062363. 11
- [79] M. M. Swift, B. N. Bershad, and H. M. Levy, "Improving the reliability of commodity operating systems," in *Proceedings of the 19th ACM Symposium on Operating Systems Principles (SOSP)*, ACM, 2003, pp. 207–222. doi: 10.1145/945445.945466. 12
- [80] M. M. Swift, M. Annamalai, B. N. Bershad, and H. M. Levy, "Recovering device drivers," *ACM Transactions on Computer Systems*, vol. 24, no. 4, pp. 333–360, 2006, doi: 10.1145/1189256.1189257. 13
- [81] D. E. Porter, O. S. Hofmann, C. J. Rossbach, A. Benn, and E. Witchel, "Operating System Transactions," in *Proceedings of the 22nd ACM Symposium on Operating Systems Principles (SOSP)*, ACM, 2009, pp. 161–176. doi: 10.1145/1629575.1629591. 14

- [82] O. Kiselyov and C.-chieh Shan, “Delimited Continuations in Operating Systems,” in *Modeling and Using Context (CONTEXT 2007)*, in Lecture Notes in Computer Science, vol. 4635. Springer, 2007, pp. 291–302. doi: 10.1007/978-3-540-74255-5_22. 1
- [83] E. Dolstra and A. Löh, “NixOS: a purely functional Linux distribution,” in *Proceedings of the 13th ACM SIGPLAN International Conference on Functional Programming (ICFP)*, ACM, 2008, pp. 367–378. doi: 10.1145/1411204.1411255. 2
- [84] ZIO, “ZIO: Type-safe, composable asynchronous and concurrent programming for Scala.” [Online]. Available: <https://zio.dev/> 3
- [85] Effect, “Effect: A TypeScript library for building robust applications.” [Online]. Available: <https://effect.website/> 4
- [86] G. Canti, “fp-ts: Functional programming in TypeScript.” [Online]. Available: <https://github.com/gcanti/fp-ts> 5
- [87] J. I. Brachthäuser, P. Schuster, and K. Ostermann, “Effects as capabilities: effect handlers and lightweight effect polymorphism,” *Proc. ACM Program. Lang.*, vol. 4, no. OOPSLA, 2020, doi: 10.1145/3428194. 6
- [88] J. I. Brachthäuser, P. Schuster, E. Lee, and A. Boruch-Gruszecki, “Effects, capabilities, and boxes: from scope-based reasoning to type-based reasoning and back,” *Proc. ACM Program. Lang.*, vol. 6, no. OOPSLA1, 2022, doi: 10.1145/3527320. 7
- [89] C. Heunen, R. Kaarsgaard, and M. Karvonen, “Reversible Effects as Inverse Arrows,” in *Proceedings of the Thirty-Fourth Conference on the Mathematical Foundations of Programming Semantics (MFPS XXXIV)*, in Electronic Notes in Theoretical Computer Science, vol. 341. 2018, pp. 179–199. doi: 10.1016/j.entcs.2018.11.009. 8
- [90] D. Orchard, V.-B. Liepelt, and H. Eades III, “Quantitative program reasoning with graded modal types,” *Proc. ACM Program. Lang.*, vol. 3, no. ICFP, 2019, doi: 10.1145/3341714. 9
- [91] R. Bianchini, F. Dagnino, P. Giannini, E. Zucca, and M. Servetto, “Coeffects for sharing and mutation,” *Proc. ACM Program. Lang.*, vol. 6, no. OOPSLA2, Oct. 2022, doi: 10.1145/3563319. 10
- [92] R. Bianchini, F. Dagnino, P. Giannini, and E. Zucca, “A Java-like calculus with heterogeneous coeffects,” *Theoretical Computer Science*, vol. 971, p. 114063, 2023, doi: 10.1016/j.tcs.2023.114063. 11
- [93] C. Torczon, E. Suárez Acevedo, S. Agrawal, J. Velez-Ginorio, and S. Weirich, “Effects and Coeffects in Call-by-Push-Value,” *Proc. ACM Program. Lang.*, vol. 8, no. OOPSLA2, Oct. 2024, doi: 10.1145/3689750. 12
- [94] R. Hirschfeld, P. Costanza, and O. Nierstrasz, “Context-oriented Programming,” *Journal of Object Technology*, vol. 7, no. 3, pp. 125–151, 2008, doi: 10.5381/jot.2008.7.3.a4. 13
- [95] P. Costanza and R. Hirschfeld, “Language constructs for context-oriented programming: an overview of ContextL,” in *Proceedings of the 2005 Symposium on Dynamic Languages (DLS '05)*, ACM, 2005, pp. 1–10. doi: 10.1145/1146841.1146842. 14
- [96] G. Salvaneschi, C. Ghezzi, and M. Pradella, “Context-oriented programming: A software engineering perspective,” *Journal of Systems and Software*, vol. 85, no. 8, pp. 1801–1817, 2012, doi: 10.1016/j.jss.2012.03.024. 15

- [97] G. Kiczales *et al.*, “Aspect-Oriented Programming,” in *ECOOP'97 — Object-Oriented Programming, 11th European Conference*, in Lecture Notes in Computer Science, vol. 1241. Springer, 1997, pp. 220–242. doi: 10.1007/BFb0053381. 1
- [98] G. Kiczales, E. Hilsdale, J. Hugunin, M. Kersten, J. Palm, and W. G. Griswold, “An Overview of AspectJ,” in *ECOOP 2001 — Object-Oriented Programming, 15th European Conference*, in Lecture Notes in Computer Science, vol. 2072. Springer, 2001, pp. 327–353. doi: 10.1007/3-540-45337-7_18. 2
- [99] A. Popovici, T. Gross, and G. Alonso, “Dynamic Weaving for Aspect-Oriented Programming,” in *Proceedings of the 1st International Conference on Aspect-Oriented Software Development (AOSD 2002)*, ACM, 2002, pp. 141–147. doi: 10.1145/508386.508404. 3
- [100] J. Bonér, “What Are the Key Issues for Commercial AOP Use: How Does AspectWerkz Address Them?,” in *Proceedings of the 3rd International Conference on Aspect-Oriented Software Development (AOSD 2004)*, ACM, 2004, pp. 5–6. doi: 10.1145/976270.976273. 4
- [101] M. Hicks, J. T. Moore, and S. Nettles, “Dynamic Software Updating,” in *Proceedings of the ACM SIGPLAN 2001 Conference on Programming Language Design and Implementation*, in PLDI '01. 2001, pp. 13–23. doi: 10.1145/378795.378798. 5
- [102] G. Stoye, M. Hicks, G. Bierman, P. Sewell, and I. Neamtiu, “Mutatis Mutandis: Safe and Predictable Dynamic Software Updating,” in *Proceedings of the 32nd ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, in POPL '05. 2005, pp. 183–194. doi: 10.1145/1040305.1040321. 6
- [103] C. M. Hayden, K. Saur, E. K. Smith, and M. Hicks, “Kitsune: Efficient, General-Purpose Dynamic Software Updating for C,” *ACM Trans. Program. Lang. Syst.*, vol. 36, no. 4, 2014, doi: 10.1145/2629460. 7
- [104] M. Overeem, M. Spoor, and S. Jansen, “The Dark Side of Event Sourcing: Managing Data Conversion,” in *IEEE 24th International Conference on Software Analysis, Evolution and Reengineering*, in SANER '17. 2017, pp. 193–204. doi: 10.1109/SANER.2017.7884621. 8
- [105] E. Gamma, R. Helm, R. Johnson, and J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*. Boston, MA: Addison-Wesley, 1994. 9
- [106] D. Leijen, “Algebraic Effect Handlers with Resources and Deep Finalization,” technical report MSR-TR-2018-10, Apr. 2018. [Online]. Available: <https://www.microsoft.com/en-us/research/publication/algebraic-effect-handlers-resources-deep-finalization/> 10
- [107] M. Fowler, “Event Sourcing.” 2005. 11
- [108] J. Lee, J. Ahn, and K. Yi, “React-tRace: A Semantics for Understanding React Hooks,” *Proc. ACM Program. Lang.*, vol. 9, no. OOPSLA2, pp. 471–498, 2025, doi: 10.1145/3763067. 12
- [109] N. Shavit and D. Touitou, “Software Transactional Memory,” in *Proceedings of the Fourteenth Annual ACM Symposium on Principles of Distributed Computing*, in PODC '95. 1995, pp. 204–213. doi: 10.1145/224964.224987. 13
- [110] T. Harris, S. Marlow, S. Peyton Jones, and M. Herlihy, “Composable Memory Transactions,” in *Proceedings of the Tenth ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming*, in PPOPP '05. 2005, pp. 48–60. doi: 10.1145/1065944.1065952. 14

- [111] M. Herlihy and J. E. B. Moss, "Transactional Memory: Architectural Support for Lock-Free Data Structures," in *Proceedings of the 20th Annual International Symposium on Computer Architecture*, in ISCA '93. 1993, pp. 289–300. doi: 10.1145/165123.165164. 1
- [112] R. Landauer, "Irreversibility and Heat Generation in the Computing Process," *IBM Journal of Research and Development*, vol. 5, no. 3, pp. 183–191, 1961, doi: 10.1147/rd.53.0183. 2
- [113] C. H. Bennett, "Logical Reversibility of Computation," *IBM Journal of Research and Development*, vol. 17, no. 6, pp. 525–532, 1973, doi: 10.1147/rd.176.0525. 3
- [114] T. Yokoyama and R. Glück, "A Reversible Programming Language and its Invertible Self-Interpreter," in *Proceedings of the 2007 ACM SIGPLAN Workshop on Partial Evaluation and Semantics-Based Program Manipulation*, in PEPM '07. 2007, pp. 144–153. doi: 10.1145/1244381.1244404. 4
- [115] V. Danos and J. Krivine, "Reversible Communicating Systems," in *CONCUR 2004 — Concurrency Theory, 15th International Conference*, in Lecture Notes in Computer Science, vol. 3170. Springer, 2004, pp. 292–307. doi: 10.1007/978-3-540-28644-8_19. 5
- [116] I. Phillips and I. Ulidowski, "Reversing Algebraic Process Calculi," in *Foundations of Software Science and Computation Structures, 9th International Conference (FOSSACS 2006)*, in Lecture Notes in Computer Science, vol. 3921. Springer, 2006, pp. 246–260. doi: 10.1007/11690634_17. 6
- [117] P. Wadler, "Linear Types Can Change the World!," in *Programming Concepts and Methods: Proceedings of the IFIP Working Group 2.2/2.3 Working Conference*, North-Holland, 1990, pp. 561–581. [Online]. Available: <https://homepages.inf.ed.ac.uk/wadler/papers/linear/linear.ps> 7
- [118] D. Kim and M. C. Rinard, "Verification of Semantic Commutativity Conditions and Inverse Operations on Linked Data Structures," in *Proceedings of the 32nd ACM SIGPLAN Conference on Programming Language Design and Implementation*, 2011, pp. 528–541. doi: 10.1145/1993498.1993561. 8
- [119] A. Lenharth, V. S. Adve, and S. T. King, "Recovery domains: an organizing principle for recoverable operating systems," in *Proceedings of the 14th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, ACM, 2009, pp. 49–60. doi: 10.1145/1508244.1508251. 9
- [120] C. Walls, *Spring in Action*, 6th ed. Manning Publications, 2022. [Online]. Available: <https://www.manning.com/books/spring-in-action-sixth-edition> 10
- [121] C. Escoffier, R. S. Hall, and P. Lalanda, "iPOJO: an Extensible Service-Oriented Component Framework," in *IEEE International Conference on Services Computing*, 2007, pp. 474–481. doi: 10.1109/SCC.2007.74. 11
- [122] H. Cervantes and R. S. Hall, "Autonomous Adaptation to Dynamic Availability Using a Service-Oriented Component Model," in *Proceedings of the 26th International Conference on Software Engineering*, in ICSE '04. 2004, pp. 614–623. doi: 10.1109/ICSE.2004.1317483. 12
- [123] C. Elliott and P. Hudak, "Functional Reactive Animation," in *Proceedings of the Second ACM SIGPLAN International Conference on Functional Programming*, in ICFP '97. 1997, pp. 263–273. doi: 10.1145/258948.258973. 13

- [124] G. H. Cooper and S. Krishnamurthi, “Embedding Dynamic Dataflow in a Call-by-Value Language,” in *Programming Languages and Systems (ESOP 2006)*, in Lecture Notes in Computer Science, vol. 3924. Springer, 2006, pp. 294–308. doi: 10.1007/11693024_20. 1
- [125] I. Maier and M. Odersky, “Deprecating the Observer Pattern with Scala.React,” technical report EPFL-REPORT-176887, 2012. [Online]. Available: <https://infoscience.epfl.ch/record/176887> 2
- [126] E. Bainomugisha, A. L. Carreton, T. Van Cutsem, W. De Meuter, and others, “A Survey on Reactive Programming,” *ACM Comput. Surv.*, vol. 45, no. 4, 2013, doi: 10.1145/2501654.2501666. 3
- [127] A. Margara and G. Salvaneschi, “On the Semantics of Distributed Reactive Programming: The Cost of Consistency,” *IEEE Trans. Software Eng.*, vol. 44, no. 7, pp. 689–711, 2018, doi: 10.1109/TSE.2018.2833109. 4